

11. Programmazione Dinamica e Algoritmi Greedy



Lezione 21 - @November 10, 2025

Problemi di ottimizzazione e Algoritmi greedy

Grafi pesati (sugli archi)

- $G(V, E)$ grafo
- $w : E \rightarrow \mathbb{R}$ funzione peso

Alcuni problemi

- cammini minimi
- commesso viaggiatore
- albero ricoprente minimo - grafi non orientati

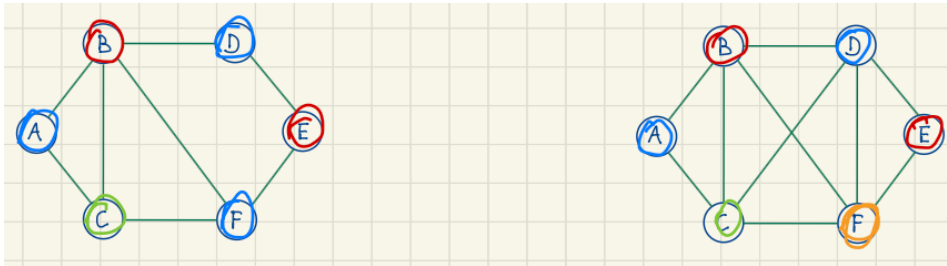
Problemi di ottimizzazione

Tra tutte le soluzioni ammissibili (soddisfano i vincoli posti dal problema) vogliamo determinarne una ottima rispetto a un dato criterio (massimizzazione o minimizzazione)

Esempio 1: Colorazione di grafi

istanza: grafo non orientato $G = (V, E)$

problema: trovare il minimo numero di colori da attribuire ai vertici di G in modo che i vertici adiacenti abbiano colori differenti



colorazione $c : V \rightarrow \text{colori} : (x, y) \in E \Rightarrow c(x) \neq c(y)$

obiettivo minimizzare $\#Im(c)$

soluzioni ammissibile: vertici adiacenti hanno colori diversi

soluzione ottima: soluzione ammissibile con h° colori minimo

Esempio 2: zaino monodimensionale

istanza: zaino di altezza h , k contenitori $c_1, c_2, \dots, c_k$ di altezze $h_1, h_2, \dots, h_k$

problema: scegliere quali contenitori collocare nello zaino, in modo da riempirlo il più possibile

soluzione ammissibile:

- sottoinsieme $S \subseteq \{c_1, c_2, \dots, c_k\}$: con $f(S) \leq h$ (vincolo) dove $f(S) = \sum_{c_i \in S} h_i$

soluzione ottima:

- soluzione ammissibile $S^* : f(S^*) \geq f(S)$ per ogni S ammissibile

Tecnica Greedy

$\mathcal{P}$ problema di ottimizzazione

- Dato C insieme di candidati
- Trovare $S \subseteq C$ ottima

strategia

- inizialmente $S \leftarrow \emptyset$
- ad ogni passo:
 - prelevo da C l'elemento x "migliore"
 - se $S \cup \{x\}$ è ammissibile aggiungo x ad S
- termino quando ho esaminato tutti i candidati

espansione

- **soluzione ammissibile**
 - la soluzione soddisfa i vincoli del problema
- **scelta dell'ottimo locale**
 - tra tutti i candidati disponibili a ogni passo si sceglie quello che al momento appare migliore
- **scelta irrevocabile**
 - le scelte fatte non vengono più messe in discussione

schema

```
ALGORITMO greedy(insieme C) → soluzione
  S ← ∅
  WHILE C ≠ ∅ DO
    x ← seleziona(C)      // elemento considerato migliore al momento
    C ← C \ {x}
    IF S ∪ {x} è ammissibile THEN
      S ← S ∪ {x}
  RETURN S
```

Zaino monodimensionale con tecnica greedy

Sia $C = \{c_1, c_2, \dots, c_k\}$ candidati - contenitori

strategia greedy: ispezioniamo i contenitori in ordine di altezza dal più alto al più basso

"seleziona": scegli il contenitore più alto

Zaino con valori (sacco del ladro)

istanza sacco che può portare al massimo peso P ; k oggetti $c_1, c_2, \dots, c_k$ di peso $p_1, p_2, \dots, p_k$ e valore $v_1, v_2, \dots, v_k$ rispettivamente

problema scegliere quali oggetti collocare nel sacco, in modo da massimizzare il valore totale, evitando che il sacco si rompa

obiettivo guadagnare il più possibile

vincolo non superare la capacità del sacco

Sia $C = \{c_1, c_2, \dots, c_k\}$ insieme dei candidati

Sottoinsieme $S \subseteq C$: $val(S) = \sum_{c_i \in S} v_i$ $peso(S) = \sum_{c_i \in S} p_i$

soluzione ammissibile

- $S \subseteq C$ è ammissibile se $peso(S) \leq P$
- $S^* \subseteq C$ è ottima se:

- 1. S^* è ammissibile
- 2. $\forall S$ ammissibile $val(S) \leq val(S^*)$



Lezione 22 - @November 12, 2025

Albero ricoprente minimo

- grafi non orientati, connessi, privi di cicli

proprietà: $G = (V, E)$ non orientato e connesso è un albero $\iff \#E = \#V - 1$

foresta: **insieme di alberi**

Albero di supporto o ricoprente - spanning tree

definizione lezioni precedenti

Albero ricoprente minimo

Problema: Dato $G = (V, E)$ non orientato connesso con una funzione peso $w : E \rightarrow \mathbb{R}$ trovare un albero ricoprente $T = (V, E_T)$ di peso minimo

Dato $G' = (V', E') : w(G') = \sum_{e \in E'} w(e)$

$T(V, E_T)$ è una soluzione ottima del problema se:

- $E_T \subseteq E, T$ è un albero
- $\forall T' = (V, E_{T'})$ se T' è un albero ricoprente $\Rightarrow w(T) \leq w(T')$

ricerca esaustiva:

$$\binom{m}{n-1} \approx m^n$$

Strategia greedy

inizialmente:

- $T = (V, \emptyset)$
- $C = E$ insieme dei candidati

ad ogni passo

- prelevo da C l'arco di peso minimo lo aggiungo a T se non forma cicli con gli archi già scelti

ALGORITMO *greedy*(insieme C) $\rightarrow$ soluzione
 $S \leftarrow \emptyset$
WHILE $C \neq \emptyset$ **DO**
 $x \leftarrow \text{seleziona}(C)$
 $C \leftarrow C \setminus \{x\}$
 IF $S \cup \{x\}$ è ammissibile **THEN**
 $S \leftarrow S \cup \{x\}$
RETURN S

ALGORITMO *Kruskal*(Grafo $G = (V, E, \omega)$) $\rightarrow$ albero
 ordina E in maniera non decrescente in base ai pesi
 $T \leftarrow (V, \emptyset)$
 FOR EACH $(x, y) \in E$ secondo l'ordine **DO**
 IF x e y non sono connessi in T **THEN**
 aggiungi a T l'arco (x, y)
 RETURN T

Algoritmo di Kruskal: correttezza



teorema: l'algoritmo di Kruskal trova un albero ricoprente di peso minimo

dim: T_k : albero ricoprente determinato dall'algoritmo di Kruskal

T_0 : albero ricoprente minimo con il maggior numero di archi in comune con T_k

Per assurdo $T_k \neq T_0$

Sia (x, y) il primo arco (nell'ordine di Kruskal) : (x, y) è in T_k ma non in T_0

Poiché T_0 è un albero, i vertici x, y sono connessi da un cammino ρ in T_0 . Pertanto aggiungendo (x, y) a T_0 si otterrebbe un ciclo.

$\Rightarrow \exists$ un arco (u, v) su tale cammino ρ con $(u, v) \in E \setminus E_T$

Poiché per Kruskal procede in ordine di peso, ha scelto (x, y) e non (u, v) si ha $\omega(x, y) \leq \omega(u, v)$

Consideriamo ora $\hat{T} = (V, \hat{E})$ con $\hat{E} = E_0 \setminus \{(u, v) \cup (x, y)\}$. Possiamo osservare che anche $\hat{T}$ è un albero ricoprente di G . Inoltre:

$$\omega(\hat{T}) \leq \omega(T_0) - \omega(u, v) + \omega(x, y) \leq \omega(T_0)$$

Dunque $\hat{T}$ è un albero ricoprente che ha lo stesso peso di T_0 (che è minimo) e ha un arco in più di T_0 in comune con T . Questo contraddice l'ipotesi iniziale su T_0 (albero ricoprente minimo *con il maggior numero di archi in comune con T*). Pertanto $T_0 = T$.

Implementazione

- è utile rappresentare il grafo come una *lista di archi*, che può essere memorizzata direttamente in un array sul quale applicare uno degli algoritmi di ordinamento (chiave: peso)
 - Possiamo considerare una partizione dell'insieme dei vertici V , in cui due vertici appartengono allo stesso insieme della partizione $\iff$ sono connessi in T . In altre parole, ogni elemento della partizione rappresenta una componente connessa di T e corrisponde un albero della foresta T
1. Inizialmente ogni vertice di V costituisce un singolo insieme della partizione (T non contiene archi e dunque non ci sono vertici connessi fra loro).
 2. Quando esaminiamo un arco (x, y) , due possibilità:
 - Se x, y appartengono allo stesso elemento della partizione $\rightarrow$ sono già connessi in T , in questo caso l'arco (x, y) non viene aggiunto a T (si creerebbe un ciclo)
 - Se x, y appartengono a elementi diversi della partizione $\rightarrow$ non sono connessi: aggiungendo l'arco (x, y) a T rendiamo ciascun vertice dell'elemento della partizione a cui appartiene x connesso a ciascun vertice dell'elemento a cui appartiene y , cioè rendiamo le due componenti connesse a cui appartengono x, y un'unica componente connessa. Pertanto, oltre ad aggiungere l'arco (x, y) all'insieme T , fondiamo i due elementi della partizione.

La partizione può essere rappresentata mediante le strutture *Union-Find*. Per verificare se x, y appartengono allo stesso insieme della partizione è sufficiente confrontare i risultati di `find(x)` e `find(y)`. Per unire due elementi dalla partizione utilizziamo `union`.

ALGORITMO *Kruskal*(Grafo $G = (V, E, \omega)$) $\rightarrow$ albero

ordina E in maniera non decrescente in base ai pesi // lista di archi

$T \leftarrow (V, \emptyset)$

Sia P una partizione inizialmente vuota

FOREACH $v \in V$ **DO** $P.makeSet(v)$

FOREACH $(x, y) \in E$ *secondo l'ordine* **DO**

$T_x \leftarrow P.find(x)$

$T_y \leftarrow P.find(y)$

IF $T_x \neq T_y$ **THEN**

$P.union(T_x, T_y)$

aggiungi a T l'arco (x, y)

RETURN T

Analisi dei tempi

- Ordinamento di E :
Possiamo utilizzare l'algoritmo `heapSort` in tempo $O(m \log m)$.
- Operazioni union/find:
supponendo di utilizzare la tecnica `QuickUnion` con bilanciamento in altezza, l'operazione `MakeSet` viene svolta in tempo costante, `Find` in tempo $O(\log n)$ e `Union` in tempo costante, dove n è il numero di elementi presenti complessivamente nell'insieme della partizione.

L'algoritmo effettua le seguenti operazioni:

- n operazioni di `makeset` $\rightarrow O(n)$
- $2m$ operazioni di `find` (2 per ogni arco) $\rightarrow O(m \log n)$
- $n - 1$ operazioni `union` (una per ogni nuovo arco aggiunto, *un albero con n vertici contiene $n - 1$ archi*) $\rightarrow O(n)$

Sommando i tempi, osservando che $n - 1 \leq m \leq n^2$, ed essendo G connesso otteniamo che:

$$T(n) = O(m \log m) + O(n) + O(m \log m) = O(m \log n^2) + O(m) + O(m \log n)$$

In caso di pesi interi si può usare come algoritmo di ordinamento `radixSort` ed impiegando la compressione di cammino per le operazioni legate alle partizione si ottiene un tempo di totale di $O(m \log^* n)$

Algoritmo di Prim

```

ALGORITMO Prim(Grafo  $G = (V, E, w)$ )  $\rightarrow$  albero
   $T \leftarrow (V_T \leftarrow \{s\}, E_T \leftarrow \emptyset)$  // albero formato da un solo vertice  $s$  qualsiasi
  WHILE  $T$  ha meno di  $n$  vertici DO
    sia  $(x, y) \in E$  di peso minimo con  $x \in V_T$  e  $y \notin V_T$ 
     $V_T \leftarrow V_T \cup \{y\}$ 
     $E_T \leftarrow E_T \cup \{(x, y)\}$ 
  RETURN  $T$ 

```



Lezione 23 - @November 14, 2025

strategia greedy alternativa,

inizialmente:

T : albero costituito da un unico vertice

ad ogni passo:

si espande T aggiungendo l'arco `(x,y)` di peso minimo con un vertice in T e l'altro non in T

Prim vs Kruskal

sono strategie greedy differenti

- Kruskal
 - soluzione parziale: foresta di alberi con insieme di vertici V
 - inizialmente: tutti i vertici, nessun arco
- Prim
 - soluzione parziale: albero $T = (V_T, E_T)$ con $V_T \subseteq V, E_T \subseteq E \cap (V_T \times V_T)$
 - inizialmente: albero formato da un unico vertice

Implementazione

Ad ogni passo per ogni vertice v non ancora in T consideriamo:

$d[v]$ = minimo peso degli archi tra v e vertici in T (∞ se non ce ne sono)

$$d[v] = \{\omega(u, v) : u \in V_T\} \cup \{\infty\}$$

$vicino[v]$ = vertice u in T tale che $d[v] = \omega(u, v)$

Osservazioni

- algoritmo greedy $\rightarrow$ sceglie il migliore fra gli archi di “frontiera”

vari livelli di astrazione con cui si può vedere l'algoritmo

Implementazione con lista di adiacenza

Costa poco il **FOREACH**

1. **riempi coda**, sapendo che la coda è rappresentata tramite un vettore posizionale, equivale a **riempi array** **costo** $O(n)$
2. **ciclo do-while** n iterazioni
 - a. **deleteMin** costo singolo $O(\log n)$ n volte $\rightarrow O(n \log n)$
 - b. costante n volte $O(n)$
 - c. intestazione del ciclo lista di adiacenza: $\delta(y)$ iterazioni $\rightarrow$ totale $2m$, tempo $O(m)$
 - d. **changeKey** $O(\log n)$ costo singolo, al massimo m volte $\rightarrow O(m \log n)$

Quindi:

$$O(n \log n) + O(m \log n) = O(m \log n)$$

Sapendo che $n - 1 \leq m \leq n^2$



Lezione 24 - @November 17, 2025

Calcolo dei numeri di Fibonacci v1

```
ALGORITMO Fibonacci(intero n) -> intero
  IF n=1 OR n=2 THEN RETURN 1
  ELSE RETURN Fibonacci(n-1)+Fibonacci(n-2)
```

numero di linee di codice eseguite:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2 + T(n-1) + T(n-2) & \text{altrimenti} \end{cases}$$

»

$$\text{Con } \phi = \frac{1 + \sqrt{5}}{2}$$

Numeri di fibonacci v2

```
ALGORITMO Fibonacci(intero n) -> intero n
  Sia Fib[1...n] un array di interi
  Fib[1] <- 1
  Fib[2] <- 2
  FOR i <- 3 TO n DO
```

```
Fib[i] <- Fib[i-1]+Fib[i-2]
RETURN Fib[n]
```

Programmazione dinamica

cammino di valore minimo su matrici $n \times n$

Problema:

Data una matrice $n \times n$ di interi determinare un "cammino" dalla prima all'ultima colonna di valore minimo

Cammino

inizio: posizione qualunque della prima colonna

da una posizione: si può raggiungere la posizione nella colonna successiva:

fine: posizione qualunque dell'ultima colonna

Valore

somma dei valori lungo il cammino

Ricerca esaustiva: esaminiamo tutti i possibili cammini

1^a colonna: n scelte

2^a colonna: ≥ 2 scelte

...

n -esima colonna "

→ almeno $n \cdot 2 \cdot 2 \cdot \dots \cdot 2 = n2^{n+1}$ scelte → numero **esponenziale** di scelte!

Programmazione dinamica

$\mathcal{P}$: problema del cammino di valore minimo in una matrice $n \times n$

Per $i, j = 1, \dots, n$ considero il problema:

$\mathcal{P}(i, j)$: trovare il cammino minimo che inizia nella colonna 1 e termina nella posizione (i, j)

La soluzione di $\mathcal{P}$ si ricava dalle soluzioni di $\mathcal{P}(1, n), \mathcal{P}(2, n), \dots, \mathcal{P}(n, n)$

matrice $\mathbf{c}$ dei risultati dei problemi $\mathcal{P}(i, j)$

$\mathbf{c}[i, j]$ = valore minimo che inizia nella colonna 1 e termina nella posizione (i, j)

il valore del cammino minimo sulla matrice è

»

```
ALGORITMO camminoMinimo(matrice  $M[1 \dots n, 1 \dots n]$ )  $\rightarrow$  intero
  Sia  $C[1 \dots n, 1 \dots n]$  una matrice      // (di interi)

  FOR  $i \leftarrow 1$  TO  $n$  DO
     $C[i, 1] \leftarrow M[i, 1]$ 

  FOR  $j \leftarrow 2$  TO  $n$  DO           // colonne
    FOR  $i \leftarrow 1$  TO  $n$  DO       // righe
       $min \leftarrow C[i, j - 1]$ 
      IF  $i > 1$  AND  $C[i - 1, j - 1] < min$  THEN  $min \leftarrow C[i - 1, j - 1]$ 
      IF  $i < n$  AND  $C[i + 1, j - 1] < min$  THEN  $min \leftarrow C[i + 1, j - 1]$ 
       $C[i, j] \leftarrow M[i, j] + min$ 

   $min \leftarrow C[1, n]$ 
  FOR  $i \leftarrow 2$  TO  $n$  DO
    IF  $C[i, n] < min$  THEN  $min \leftarrow C[i, n]$ 
  RETURN  $min$ 
```

Tempi:

	Costo singolo	costo per n
Assegnamento	$O(1)$	$O(n)$
Operazioni nei due <code>for</code> annidati	$O(1)$	$O(n^2)$
<code>for</code> finale	$O(n)$	

$\rightarrow$ Tempo totale $O(n^2)$

È possibile ricostruire il cammino “andando a ritroso”

Soluzione di un problema a partire da sottoproblemi più semplici

- Si individuano sottoproblemi del problema dato
- Si risolvono i sottoproblemi a partire dai dati più semplici
 - Sottoproblemi base: soluzione immediata $\rightarrow$ memorizzata in tabella
 - altri sottoproblemi: soluzione ottenuta utilizzando le soluzioni di sottoproblemi risolti in precedenza $\rightarrow$ consultazione e aggiornamento tabella
- soluzione finale: ricavata dalle soluzione dei sottoproblemi $\rightarrow$ funzione degli elementi della tabella

Cammini minimi (nei grafi)

$G = (V, E)$ grafo orientato

$\omega : E \rightarrow \mathbb{R}$ funzione peso

$\pi = \langle v_0, v_1, \dots, v_k \rangle$ cammino da v_0 a v_k

$$\omega(\pi) = \sum_{i=1}^k \omega(v_{i-1}, v_i) \quad \text{peso del cammino}$$

π è un cammino minimo da `x` a `y` se:

- π è un cammino da `x` a `y`
- $\forall \pi'$ cammino da `x` a `y` $\omega(\pi) \leq \omega(\pi')$

Proprietà - principio di ottimalità

- Se π è un cammino da `x` a `y` allora

π' è un cammino minimo da x a v

π'' è un cammino minimo da v a y

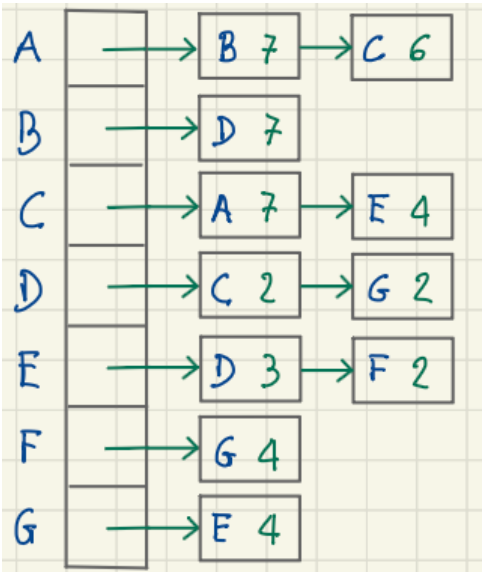
- se tutti i pesi sono positivi, allora ogni cammino minimo è un cammino semplice.

se ci sono dei vertici ripetuti, si può eliminare il percorso tra i due vertici.

- se ci sono pesi negativi ma non ci sono cicli negativi allora se x e y esiste un cammino allora tra x e y esiste un cammino minimo semplice

Rappresentazioni dei grafi pesati

1. Lista di adiacenza con pesi



2. Matrice dei pesi

	A	B	C	D	E	F	G
A	∞	7	6	∞	∞	∞	∞
B	∞	∞	∞	7	∞	∞	∞
C	7	∞	∞	∞	4	∞	∞
D	∞	∞	2	∞	∞	∞	2
E	∞	∞	∞	3	∞	2	∞
F	∞	∞	∞	∞	∞	∞	4
G	∞	∞	∞	∞	4	∞	∞